

Programming Assignment Report

Design and Analysis of Algorithms

Name: Nimrah.N

Registration Number: 2411021061174

Problem Name: Minimum Spanning Tree using Prim's Algorithm

Problem Statement

Given a graph consisting of several edges connecting its nodes, find a subgraph with the following properties:

- The subgraph contains all the nodes present in the original graph.
- The subgraph has the minimum possible total edge weight among all such subgraphs.
- There is exactly one unique path between any two nodes in the subgraph.

This type of subgraph is known as a Minimum Spanning Tree (MST).

One specific node is provided as the starting point, and the tree must be constructed using Prim's Algorithm beginning from this node.

Your task is to determine the total weight (sum of edge weights) of the resulting Minimum Spanning Tree.

Problem HackerRank Link

<https://www.hackerrank.com/challenges/primsmstsub>

Github Submission Link

https://github.com/Nimrah456/DAA_LAB/blob/main/Prims%20algorithm%20assignmnet.java

Solution Steps

Prim's algorithm is a greedy algorithm used to find the Minimum Spanning Tree of a weighted graph.

Step 1: Build the Graph

- Read all edges from the input
- For each edge (u, v, w) , add both directions since the graph is undirected
- Store neighbors of each node in an adjacency list

Step 2: Initialize Data Structures

- Create a visited array to track nodes included in MST
- Create a priority queue (min heap)
- Insert the starting node with weight 0
- Initialize `totalWeight = 0`

Step 3: Process Nodes

- Extract node with minimum edge weight from the priority queue
- If node already visited, skip
- Otherwise mark visited and add weight to MST total

Step 4: Explore Neighbors

- For the selected node, check all neighbors
- If neighbor not visited, add it to the priority queue with edge weight

Step 5: Repeat

- Continue until priority queue becomes empty
- Gradually include all nodes in the MST

Step 6: Return Result

- The `totalWeight` represents the weight of the Minimum Spanning Tree
- Return `totalWeight` as final result

Complete Java Code

```
public static int prims(int n, List<List<Integer>> edges, int start) {
```

```
    List<List<int[]>> adj = new ArrayList<>();
```

```
    for (int i = 0; i <= n; i++) {  
        adj.add(new ArrayList<>());  
    }
```

```
    for (List<Integer> e : edges) {  
        int u = e.get(0);  
        int v = e.get(1);  
        int w = e.get(2);  
  
        adj.get(u).add(new int[]{v, w});  
        adj.get(v).add(new int[]{u, w});  
    }
```

```
    boolean[] visited = new boolean[n + 1];
```

```
    PriorityQueue<int[]> pq = new PriorityQueue<>(  
        (a, b) -> a[1] - b[1]  
    );
```

```
    pq.add(new int[]{start, 0});
```

```
    int totalWeight = 0;
```

```
    while (!pq.isEmpty()) {
```

```
        int[] curr = pq.poll();  
        int node = curr[0];
```

```

    int weight = curr[1];

    if (visited[node]) continue;

    visited[node] = true;
    totalWeight += weight;

    for (int[] neighbor : adj.get(node)) {
        if (!visited[neighbor[0]]) {
            pq.add(new int[]{neighbor[0], neighbor[1]});
        }
    }
}

return totalWeight;
}

```

Observation / Solution

- Prim's algorithm constructs a Minimum Spanning Tree from a weighted graph.
- It always selects the smallest edge connecting a visited node to an unvisited node.
- A priority queue ensures the smallest edge is selected efficiently.
- A visited array prevents processing the same node multiple times.
- The algorithm expands the tree until all vertices are included.
- The result is the total weight of the selected edges in the MST.

Time Complexity

Time Complexity: $O(E \log V)$

Where V is the number of vertices and E is the number of edges.